

Pareto processes

Clément Dombry, Juliette Legrand and Thomas Opitz

Contents

1	Introduction	1
2	Numerical experiments	1
2.1	Site conditioning risk functional	2
2.2	Sum risk functional	3
2.3	Maximum risk functional	5
2.4	Minimum risk functional	5
3	Application to significant wave heights	7
3.1	Marginal standardization	7
3.2	Extremal dependance	8
3.3	Model validation with the extremogram	10
4	References	11
5	Additional informations	11

1 Introduction

This document is a supplementary file for the Chapter *Pareto processes* from *Handbook of Statistics of Extremes*. It presents additional simulation frameworks and the complete code for the application example.

Note that some external sources that have proved very valuable are the R vignette from Léo Belzile presenting some of the uses of the `mev` package (<https://lbelzile.github.io/mev/article/s/04-simulation.html#exercice>), and a R tutorial from Raphael De Fondeville on the `mvPot` package, available here: https://github.com/r-fndv/mvPot_tutorial/blob/master/EVA%202019%20Workshop.ipynb

2 Numerical experiments

In the following, we propose several implementations for simulations of r-Pareto processes, given different risk functional r . For each risk functional, we propose an implementation “by

hand”, *i.e.* following Example 3 of the Chapter, and compare it with the `mev` package.

In addition to the choice of r , simulations also involves the choice of a variogram (or semivariogram) function for the underlying process, as explained in the Chapter. We consider here only the power variogram for different parameters.

For more details on the codes, please refer to the Chapter.

The numerical experiments will be performed on a regular grid 20×20 , denoted `grid`. We also need the matrix of distances `distmat`, which comprises the distances between each pair of grid cells.

```
grid = as.matrix(expand.grid(1:20,1:20))
distmat = as.matrix(dist(grid))
```

2.1 Site conditioning risk functional

The following compare simulations using the power variogram for different set of parameters, using the methodology presented in Example 3 as presented in the Chapter. Results are depicted in Figure 1.

```
s0 = 110
vario_power = function(h,beta=1,alpha=1.5){(h/beta)^alpha}
gamma = vario_power(distmat)

G = geoR::grf(grid = grid, cov.pars = c(1, 1.5),cov.model="power")$data
Y = exp(G-G[s0]-gamma[,s0])
R = evd::rgpd(n = 1, loc = 1, scale = 1, shape = 1)
Z1 = R*Y

gamma = vario_power(distmat,beta=1.5,alpha=0.8)
G = geoR::grf(grid = grid, cov.pars = c((1/1.5)^0.8, 0.8),
              cov.model="power")$data
Y = exp(G-G[s0]-gamma[,s0])
Z2 = R*Y
```

We can also consider the function `rparp` from the R package `mev`. The user can specify the risk functional with the argument `riskf`, where currently available risks includes maxima, minima, geometric average, location-conditioning value and L^2 norm, and a specific dependence model using the argument `model` (*e.g.*, “br” for the Brown-Resnick model) along with a user-defined semivariogram (argument `vario`) or a parameter matrix (argument `sigma`).

In the following we consider the site conditioning risk functional for a Brown-Resnick model with the power variogram, and compare it with the previous approach (see Figure 2).

```
s0 = 350
mev_samp = mev::rparp(n = 1,shape = 1,riskf = "site",siteindex=s0,
                     vario = vario_power,beta=0.5,alpha=0.9,
```

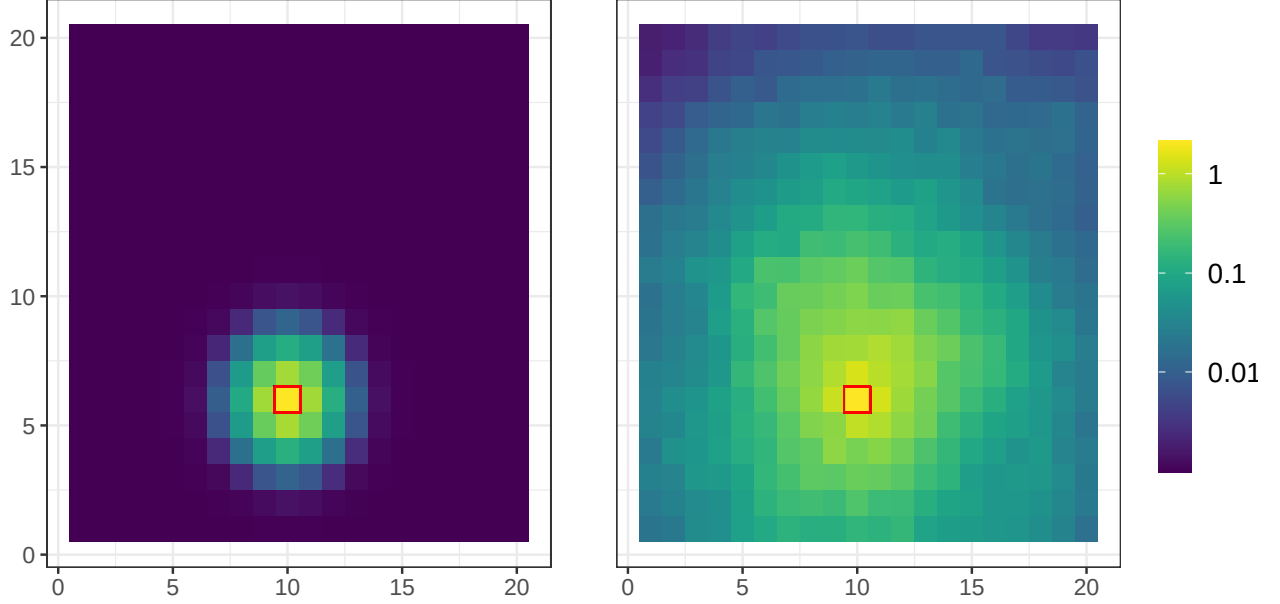


Figure 1: One realisation on logarithmic scale of a r-Pareto process with $r(f) = f(s_0)$, using a Brown-Resnick model and a power variogram defined as $\gamma(h) = (h/\beta)^\alpha$. Left panel: $\beta = 1$, $\alpha = 1.5$. Right panel: $\beta = 1.5$, $\alpha = 0.8$. The red square highlights the conditioning site s_0 .

```
coord=grid,model = "br")

gamma = vario_power(distmat,beta=0.5,alpha=0.9)
G = geoR::grf(grid = grid, cov.pars = c((1/0.5)^0.9, 0.9),
               cov.model="power")$data
Y = exp(G-G[s0]-gamma[,s0])
R = evd::rgpd(n = 1, loc = 1, scale = 1, shape = 1)
Z1 = R*Y
```

2.2 Sum risk functional

Following Proposition 1 and since simulation given the site conditioning risk functional is feasible, simulation of r-Pareto processes with $r = \sum_{i=1}^K f(s_k)$ can be performed as follow. Again we compare with the built-in function of the package `mev`, see Figure 3.

```
s0 = sample(1:nrow(grid),1)
G = geoR::grf(grid = grid, cov.pars = c(1, 1.5),cov.model="power")$data
gamma = vario_power(distmat)
Y = exp(G-G[s0]-gamma[,s0])
R = evd::rgpd(n = 1, loc = 1, scale = 1, shape = 1)
Z1 = R*Y/sum(Y)

mev_samp = mev::rparp(n = 1,shape = 1,riskf = "sum",vario = vario_power,
                      coord=grid,model = "br")
```

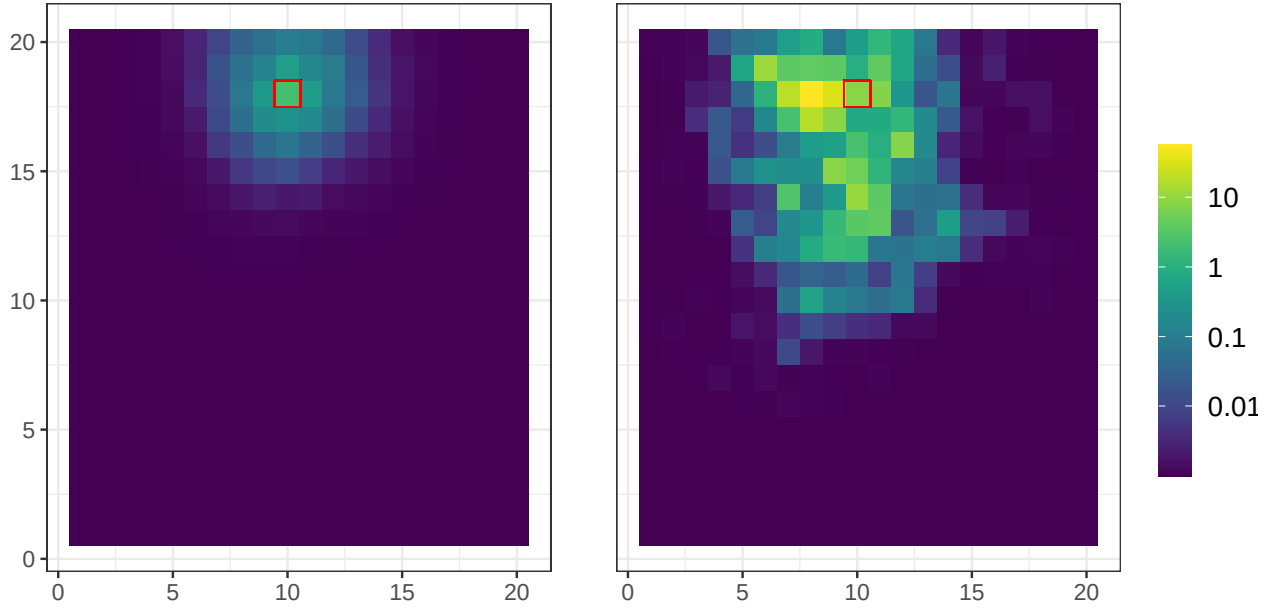


Figure 2: One realisation on logarithmic scale of a r-Pareto process with $r(f) = f(s_0)$, using a Brown-Resnick model and a power variogram defined as $\gamma(h) = (h/\beta)^\alpha$ with $\beta = 0.5$, $\alpha = 0.9$. Left panel: by hand. Right panel: using mev package. The red square highlights the conditioning site s_0 .

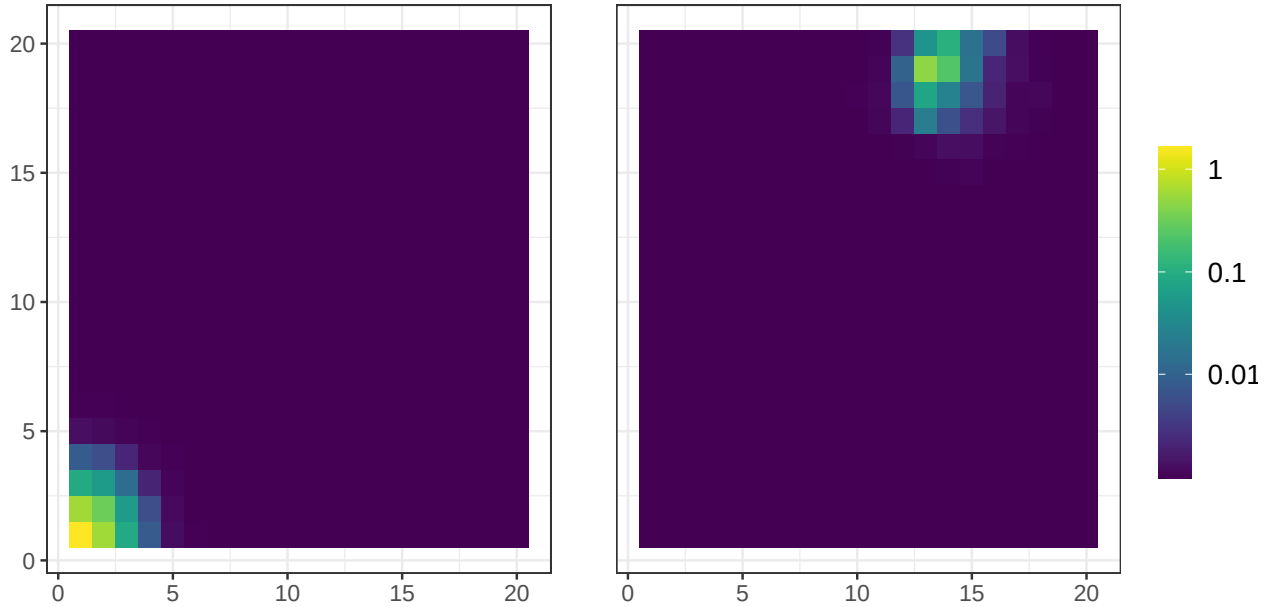


Figure 3: One realisation on logarithmic scale of a r-Pareto process with $r(f) = \sum_{i=1}^K f(s_k)$, using a Brown-Resnick model and a power variogram defined as $\gamma(h) = (h/\beta)^\alpha$, with $\beta = 1$, $\alpha = 1.5$. Left panel: by hand. Right panel: using the mev package.

2.3 Maximum risk functional

Following Proposition 2, and since we are able to simulate Pareto processes with mean risk functional, we can also perform simulations of Pareto processes using a rejection-sampling approach for maximum risk functional. Results are presented in Figure 4.

```
Z1 = rep(0,nrow(grid))
gamma = vario_power(distmat)
while(max(Z1)<nrow(grid)){
  s0 = sample(1:nrow(grid),1)
  G = geoR::grf(grid = grid, cov.pars = c(1, 1.5),cov.model="power")$data
  Y = exp(G-G[s0]-gamma[,s0])
  R = evd::rgpd(n = 1, loc = 1, scale = 1, shape = 1)
  Z1 = R*Y/mean(Y)
}
Z2 = Z1/nrow(grid)

mev_samp = mev::rparp(n = 5,shape = 1,riskf = "max",vario = vario_power,
  coord=grid, model = "br")
```

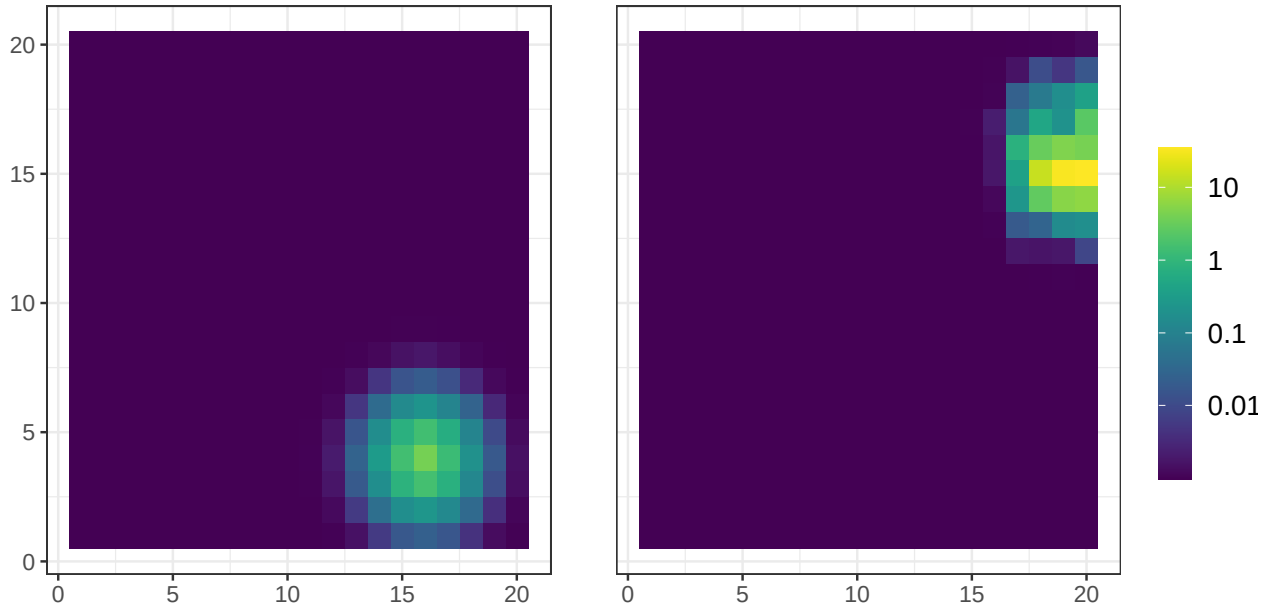


Figure 4: One realisation on logarithmic scale of a r-Pareto process with $r(X) = \max_{s \in S} f(s)$, using a Brown-Resnick model and a power variogram defined as $\gamma(h) = (h/\beta)^\alpha$ (with $\beta = 1$, $\alpha = 1.5$). Left panel: using the procedure described in Proposition 2. Right panel: using the R package mev, also based on rejection sampling.

2.4 Minimum risk functional

With minimum risk functional, codes can be computationally heavy since we use the rejection-sampling approach. The example below is therefore run on a smaller grid 10×10 , see Figure

5. Similarly, simulation with the `mev` package can take some time. To avoid this, one can use the function `rpapcs` instead, which uses composition sampling (note that this function takes as argument the variogram matrix, defined as $\Lambda_{i,j} = \gamma(s_i, s_j)/4$ for sites $s_i, s_j \in \mathcal{S}$). The `rpapcs` function is implemented only for $r = \min$ and $r = \max$.

```
grid = as.matrix(expand.grid(1:10,1:10))
distmat = as.matrix(dist(grid))
gamma = vario_power(distmat,beta=2,alpha=1.2)

Z1 = rep(0,nrow(grid))
while(min(Z1)<1){
  s0 = sample(1:nrow(grid),1)
  G = geoR::grf(grid = grid, cov.pars = c((1/2)^1.2,1.2),cov.model="power")$data
  Y = exp(G-G[s0]-gamma[,s0])
  R = evd::rgpd(n = 1, loc = 1, scale = 1, shape = 1)
  Z1 = R*Y/mean(Y)
}
Z2 = Z1

mev_samp = mev::rpapcs(n = 1,shape = 1,riskf = "min",Lambda = gamma/4,model = "br")
```

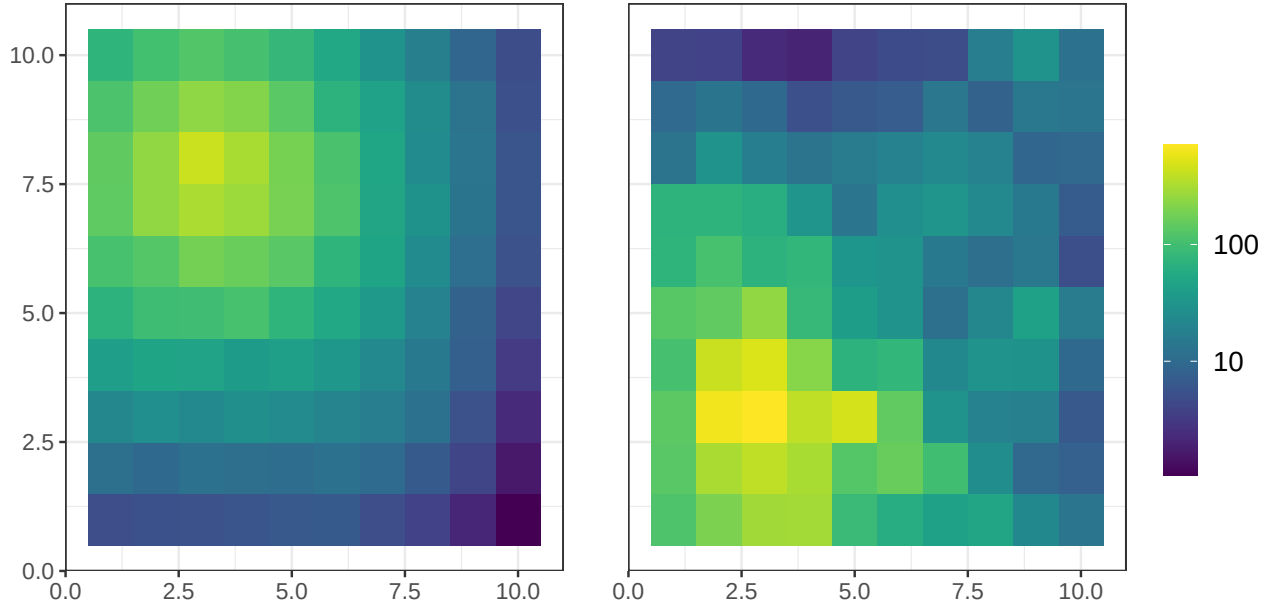


Figure 5: One realisation on logarithmic scale of a r -Pareto process with $r(X) = \min_{s \in \mathcal{S}} f(s)$, using a Brown-Resnick model and a power variogram defined as $\gamma(h) = (h/\beta)^\alpha$ (with $\beta = 2$, $\alpha = 1.2$). Left panel: following Proposition 2. Right panel: with the `mev` package.

3 Application to significant wave heights

For the application, we follow the same procedure as in the R tutorial of Raphael De Fondeville, available here https://github.com/r-fndv/mvPot_tutorial/blob/master/EVA%202019%20Workshop.ipynb. Therefore, some part of the codes are identical to the ones in the tutorial previously mentioned.

We first load the data. A complete description can be found in the `readme.txt` file.

```
load("waveheight.RData")
```

Some transformations of the coordinates need to be performed prior to the modelization. For this we use the R package `sp` to transform longitude/latitude coordinates to UTM coordinates system, in km. Given the spatial data at hand, attention must be paid to the projection region defined with the function `CRS`.

```
library(sp)
# Create coordinates object
coords = data.frame("X" = Locations$Longitude, "Y" = Locations$Latitude)
coordinates(coords) = c("X", "Y")
# Define system of coordinates
proj4string(coords) = CRS("+proj=longlat +datum=WGS84")
# Project lon/lat coordinates to UTM (m) system of coordinates
projected_coordinates = spTransform(coords,
                                     CRS("+proj=utm +zone=30 ellps=WGS84"))
# Rescale to have new coordinates in km
projected_coordinates@coords = projected_coordinates@coords / 1000
```

3.1 Marginal standardization

Before modelling the extremal dependence structure, data are first standardized to common margins. Here to standard Pareto distribution (but could also be to common unit Fréchet distributions).

For this we define a local quantile vector `Quantiles`, with quantile level 0.95.

```
quantileLevel = 0.95
Quantiles = as.vector(apply(hs_dat, FUN = quantile, probs = quantileLevel,
                           MARGIN = 2))
```

Above these quantiles, a GPD is fitted at each location, here using the function `fit.gpd` from the R package `mev`.

```
# Parameters for generalized Pareto distributions
scales = rep(NA, ncol(hs_dat))
shapes = rep(NA, ncol(hs_dat))

# Local GP fit above the 0.95 quantile
```

```

for(i in 1:length(scales)){
  fitGP = mev::fit.gpd(hs_dat[, i], threshold = Quantiles[i])
  # Store estimation results
  scales[i] = fitGP$estimate[1]
  shapes[i] = fitGP$estimate[2]
}

# Marginal standardization ----
# Create object for normalized data
normalized_database = matrix(NA, nrow = nrow(hs_dat), ncol = ncol(hs_dat))
# Standarize to unit Pareto
for(i in 1:length(scales)){
  # Compute local empirical CDF
  empiricalCdf = ecdf(hs_dat[,i])
  # Use empirical cdf below the threshold
  normalized_database[(hs_dat[,i] <= Quantiles[i] & !is.na(hs_dat[,i])),i] =
    1 / (1 - empiricalCdf(hs_dat[(hs_dat[,i] <= Quantiles[i] &
      !is.na(hs_dat[,i])),i]))-1
  # Use estimated GP distribution above the threshold
  normalized_database[(hs_dat[,i] > Quantiles[i] & !is.na(hs_dat[,i])),i] =
    1 / ((1 - quantileLevel) * (1 + shapes[i]*(hs_dat[(hs_dat[,i] >
      Quantiles[i] &
      !is.na(hs_dat[,i])),i]-Quantiles[i])/
      scales[i]))^(-1 / shapes[i])) - 1
}

```

Figure 6 highlights the spatial variability of the estimated shape parameters, showing negative parameters, which decrease closer to the coast (which is to be expected with this type of data).

3.2 Extremal dependance

Next we model the extremal dependence structure of the marginally normalized data by fitting a r-Pareto process associated with risk functional $r(X) = \max_{s \in S} X(s)$. In addition to the choice of risk functional (often driven by the application), we can assume a parametric form for the angular process Y . A simple choice is to consider a log-Gaussian model, with power variogram $\gamma(h) = (h/\beta)^\alpha$.

The power variogram is defined as follows:

```

power_vario = function(h, scale=0.5, shape=1.5){(norm(h, type="2")/scale)^shape}

```

The estimation procedure is performed using the gradient scoring rule approach (see the Chapter for more details). Note that this approach can be applied for any risk function that is differentiable. This excludes for instance the risks functional $r(X) = \max_{s \in S} X(s)$ or $r(X) = \sup_{s \in S} X(s)$. However smooth approximations can be used, such as the p -norm

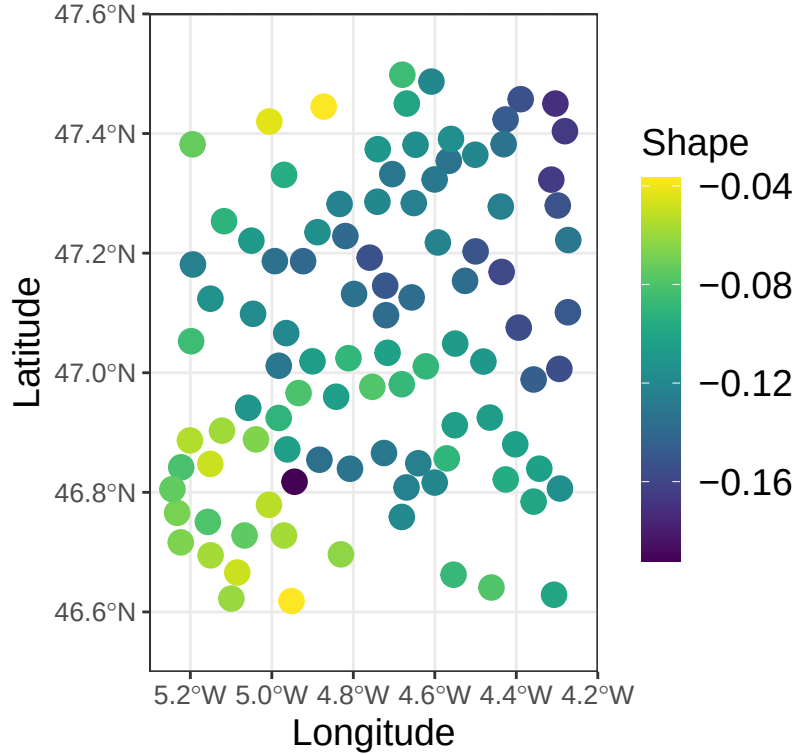


Figure 6: Estimated shape parameters from the fitted GPD at each location.

for the maximum risk functional, which is what is done below. We then define the weight function and its derivative, as explained in Section 1.3.3 and in R. de Fondeville and Davison (2018).

```
# Risk functional
riskFun = function(x,p=40){sum(x^p)^{1/p}}
# Derivative of the risk functional
dRiskFun = function(x,p=40){sum(x^(p-1))*sum(x^p)^{1/p-1}}
# Weighting function
weightFun = function(x,u){x*(1-exp(1-riskFun(x/u)))}
# Derivative of weighting function
dWeightFun = function(x, u){(1-exp(1-riskFun(x/u)))+
  x*dRiskFun(x/u)*exp(1-riskFun(x/u))}
```

Then, we compute the local maxima on the rescaled observations and define the list of observations that belongs to the exceedance region $\{x : r(x) > u\}$.

```
# Compute spatial maxima for each rescaled observations
maxs = apply(normalized_database,1, riskFun)
# Define threshold for exceedance
threshold = quantile(maxs, 0.9)
# Create matrix of exceedances for spatial maxima
spatial_accumulation_excesses = normalized_database[maxs > threshold,]
```

```
# Convert to a list for the function scoreEstimation
list_spatial_accumulation_excesses =
  lapply(1:length(spatial_accumulation_excesses[,1]), function(i){
    spatial_accumulation_excesses[i,]})
```

Then the objective function is optimized in order to obtain the variogram parameters. The objective function for the gradient scoring rule approach is implemented in the `mvPot` package with the function `scoreEstimation`.

```
# Define the objective function
objective_function = function(parameter, excesses, locations_coordinates,
                              vario, weightFun,
                              dWeightFun,
                              threshold){
  # Enforce boundaries on the variogram parameters
  if(parameter[1] < 0 | parameter[2] >= 2 | parameter[2] < 0){return(1e50)}
  variogram_model = function(h){
    power_vario(h, scale=parameter[1], shape=parameter[2])
  }
  #Compute score
  mvPot::scoreEstimation(excesses, locations_coordinates, variogram_model,
                        weightFun, dWeightFun, u = threshold)
}

# Estimate the parameter
estimates = optim(par = c(290, 1.6),
                 fn = objective_function,
                 excesses = list_spatial_accumulation_excesses,
                 locations_coordinates =
                   as.data.frame(projected_coordinates@coords),
                 vario = power_vario,
                 weightFun = weightFun,
                 dWeightFun = dWeightFun,
                 threshold = threshold,
                 control = list(maxit = 10000, trace = 0),
                 method = "Nelder-Mead")
```

The estimated parameters of the power variogram $\gamma(h) = (h/\beta)^\alpha$ are then the following:

```
## [1] "beta = 251.63 , alpha = 1.6"
```

3.3 Model validation with the extremogram

Finally, we report hereinafter the code used for Figure 7. For the empirical extremograms we use the function `extremo` from the package `mev`.

```

# Compute all possible pairwise combinations
pairs_combination = expand.grid(1:ncol(hs_dat), 1:ncol(hs_dat))

# Compute vector of distance between each pairs
distances = sqrt((projected_coordinates@coords[pairs_combination[,1],1] -
                  projected_coordinates@coords[pairs_combination[,2],1])^2 +
                  (projected_coordinates@coords[pairs_combination[,1],2] -
                  projected_coordinates@coords[pairs_combination[,2],2])^2)

# Compute orientation of each pairs
matAngles = atan((projected_coordinates@coords[pairs_combination[,1],2] -
                  projected_coordinates@coords[pairs_combination[,2],2]) /
                  (projected_coordinates@coords[pairs_combination[,1],1] -
                  projected_coordinates@coords[pairs_combination[,2],1])) *
              (180 / pi)
matAngles[is.nan(matAngles)] = 0

# Compute theoretical extremogram from variogram
extremogram_model = rep(0, length(distances))
for(i in 1:length(distances)){
  # Vector of coordinates difference between pairs
  coord_diff = c(distances[i]*cos(matAngles[i]),
                  distances[i] * sin(matAngles[i]))
  # Compute extremogram between pairs
  extremogram_model[i] =
    2*(1-pnorm(sqrt(power_vario(coord_diff, scale = estimates$par[1],
                                shape = estimates$par[2]) / 2)))
}

```

4 References

- de Fondeville, Raphael. n.d. “mvPot Tutorial.” https://github.com/r-fndv/mvPot_tutorial/blob/master/EVA%202019%20Workshop.ipynb.
- Leo R Belzile. 2023. *mev: Modelling of Extreme Values*. <https://lbelzile.github.io/mev/>.
- R. de Fondeville, and A. C. Davison. 2018. “High-Dimensional Peaks-over-Threshold Inference.” *Biometrika* 105 (3): 575–92. <https://doi.org/10.1093/biomet/asy026>.

5 Additional informations

R packages that have been used for graphical purposes are the following:

```

library(ggplot2)      # for nice graphics
library(viridis)      # for nice colors

```

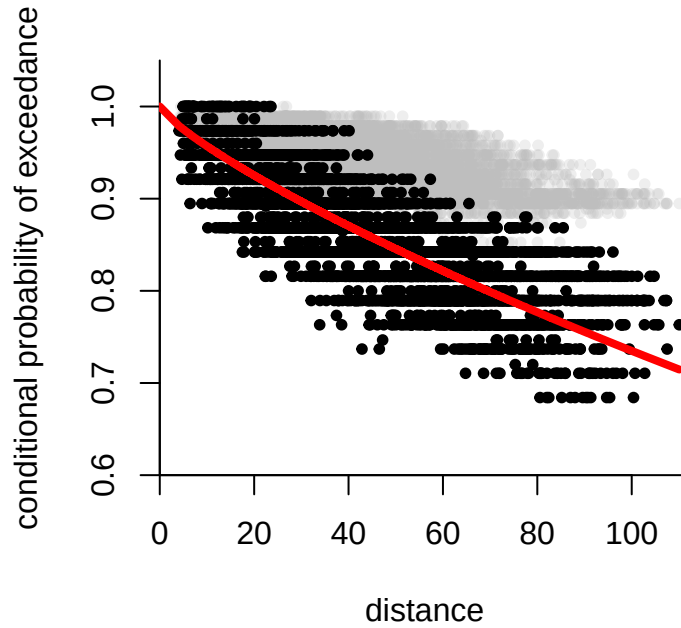


Figure 7: Empirical extremogram χ_u for $u = 0.95$ (dark dots) and $u = 0.98$ (light dots), and theoretical extremogram obtained using the estimated parameters (solid line), as functions of the transformed distance.

And session information:

```
## R version 4.2.3 (2023-03-15)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 22.04.2 LTS
##
## Matrix products: default
## BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0
##
## locale:
##  [1] LC_CTYPE=fr_FR.UTF-8      LC_NUMERIC=C
##  [3] LC_TIME=fr_FR.UTF-8      LC_COLLATE=fr_FR.UTF-8
##  [5] LC_MONETARY=fr_FR.UTF-8  LC_MESSAGES=fr_FR.UTF-8
##  [7] LC_PAPER=fr_FR.UTF-8     LC_NAME=C
##  [9] LC_ADDRESS=C             LC_TELEPHONE=C
## [11] LC_MEASUREMENT=fr_FR.UTF-8 LC_IDENTIFICATION=C
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] sp_2.1-1      viridis_0.6.2    viridisLite_0.4.2 ggplot2_3.4.4
##
```

```

## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.0      xfun_0.41          sf_1.0-14
## [4] lattice_0.20-45      tcltk_4.2.3        colorspace_2.1-0
## [7] vctr_0.6.4           generics_0.1.3     htmltools_0.5.5
## [10] mvPot_0.1.6          yaml_2.3.7         utf8_1.2.4
## [13] rlang_1.1.2          e1071_1.7-13       pillar_1.9.0
## [16] DBI_1.1.3            glue_1.6.2         withr_2.5.2
## [19] lifecycle_1.0.3      munsell_0.5.0      gtable_0.3.4
## [22] mvtnorm_1.1-3        evaluate_0.20       labeling_0.4.3
## [25] knitr_1.42           fastmap_1.1.0      class_7.3-21
## [28] fansi_1.0.5          evd_2.3-6.1        highr_0.9
## [31] Rcpp_1.0.11          KernSmooth_2.23-20 classInt_0.4-10
## [34] scales_1.2.1         alabama_2022.4-1   farver_2.1.1
## [37] gridExtra_2.3        digest_0.6.31      bookdown_0.37
## [40] dplyr_1.1.3          geoR_1.9-2         TruncatedNormal_2.2.2
## [43] numDeriv_2016.8-1.1 splancs_2.01-43    grid_4.2.3
## [46] cli_3.6.1            tools_4.2.3        magrittr_2.0.3
## [49] proxy_0.4-27         nleqslv_3.3.4      tibble_3.2.1
## [52] pkgconfig_2.0.3      MASS_7.3-58.3      randtoolbox_2.0.4
## [55] rmarkdown_2.21       rstudioapi_0.14    R6_2.5.1
## [58] mev_1.14             units_0.8-4        rngWELL_0.10-9
## [61] compiler_4.2.3

```